

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

ОТЧЁТ

о выполнении научно-исследовательской работы

по теме:

**Развёртывание кластера SLURM и моделирование потока
пользовательских заданий**

Выполнил студент группы _____ Семенов И. А.
5130201/40002

подпись

Проверил преподаватель _____ Чуватов М. В.

подпись

«_____» _____ 2026 г.

Санкт-Петербург
2026

Реферат

Отчёт посвящён развёртыванию учебного вычислительного кластера под управлением SLURM и разработке программы генерации пользовательских заданий на основе статистики реальной очереди. В ходе работы была подготовлена среда из пяти виртуальных машин на базе openSUSE Leap: управляющий узел и четыре вычислительных узла. Для узлов настроены статическая адресация, разрешение имён, SSH-доступ по ключам, общая служба аутентификации MUNGE и конфигурация SLURM.

Для моделирования нагрузки использован фрагмент набора данных SLURM, отфильтрованный по индивидуальному варианту: UID 50728 и имя задания `qe7`. По исходным данным рассчитаны распределения фактического времени выполнения и ошибки пользовательского прогноза времени. На основе полученных параметров реализован генератор пакетных заданий SLURM, который формирует сценарии с командой `sleep`, задаёт плановое время выполнения и автоматически помещает задания в очередь. Для проверки результатов подготовлен модуль анализа, строящий графики исходных и сгенерированных распределений, а также сравнение планового и фактического времени выполнения.

Ключевые слова: SLURM, MUNGE, openSUSE Leap, кластер, виртуальная машина, пакетное задание, очередь заданий, логнормальное распределение, нормальное распределение, моделирование нагрузки.

Содержание

Реферат	2
Введение	4
1 Постановка задачи	5
2 Основная часть	6
2.1 Подготовка виртуальной среды	6
2.2 Настройка сети и доступа	6
2.3 Установка MUNGE и SLURM	7
2.4 Обработка исходного набора данных	8
2.5 Аппроксимация исходных распределений	9
2.6 Генерация пакетных заданий	12
2.7 Запуск заданий и сбор фактической статистики	13
2.8 Анализ фактической ошибки прогноза	15
Заключение	18
Список использованных источников	19
Приложение А. Фрагмент конфигурации SLURM	20
Приложение Б. Пример сгенерированного задания SLURM	21
Приложение В. Код генератора пакетных заданий	22

Введение

Суперкомпьютерные системы используются для выполнения вычислительных задач, требующих значительного объёма процессорного времени и памяти. Такие системы обычно имеют кластерную архитектуру: множество вычислительных узлов объединены сетью и управляются специализированной системой диспетчеризации заданий. Одной из наиболее распространённых систем такого класса является SLURM.

Качество планирования в вычислительном кластере зависит от того, насколько точно пользователь оценивает требуемое время выполнения задания. На практике пользователь часто указывает время с запасом: фактическое время выполнения оказывается меньше запрошенного. Из-за этого диспетчер вынужден резервировать ресурсы на большее время, чем действительно необходимо, что усложняет планирование очереди и может снижать загрузку узлов.

Цель работы — развернуть учебную среду SLURM и разработать генератор пользовательских заданий, который воспроизводит свойства выбранного фрагмента реальной очереди. Для этого необходимо построить кластер из виртуальных машин, настроить сетевое взаимодействие между узлами, подготовить конфигурацию SLURM, проанализировать исходный набор данных и реализовать запуск сгенерированных заданий.

В отчёте описаны этапы подготовки виртуальных машин, настройки сетевой связности и SSH-доступа, установки MUNGE и SLURM, выбора параметров конфигурации, обработки исходного набора данных, генерации заданий и анализа результатов выполнения.

1 Постановка задачи

В рамках летней практики необходимо подготовить учебный кластер SLURM и программные средства для генерации нагрузки, основанной на статистике реальных пользовательских заданий.

Для достижения цели необходимо выполнить следующие этапы:

- создать пять виртуальных машин: один управляющий узел и четыре вычислительных узла;
- настроить статическую адресацию, разрешение имён и сетевую связность между всеми узлами;
- настроить SSH-доступ по ключам между узлами без ввода пароля;
- установить и настроить MUNGE для единой аутентификации;
- установить SLURM, сформировать конфигурационный файл через официальный конфигуратор и разложить его на все узлы;
- проверить состояние вычислительных узлов средствами SLURM;
- отфильтровать исходный набор данных по индивидуальному варианту;
- рассчитать параметры распределений фактического времени выполнения и ошибки прогноза времени;
- реализовать генератор пакетных заданий SLURM;
- запустить сгенерированные задания на кластере и собрать статистику выполнения;
- построить графики исходных, сгенерированных и фактических распределений.

Индивидуальный вариант работы определяется строкой 25 таблицы вариантов: дистрибутив openSUSE Leap, UID 50728, имя задания q_{e7} . Исходные записи набора данных фильтруются по указанному UID и имени задания.

Результатом работы должны стать настроенный кластер SLURM, исходный код генератора и анализатора, набор сгенерированных заданий, результаты выполнения на кластере и отчёт, оформленный в соответствии с ГОСТ 7.32–2017.

2 Основная часть

2.1 Подготовка виртуальной среды

Практическая часть работы выполнялась на локальной машине с использованием Oracle VM VirtualBox. В качестве модели вычислительного кластера была использована схема из пяти виртуальных машин: один управляющий узел и четыре вычислительных узла. Управляющий узел получил имя `mgmt`, вычислительные узлы получили имена `cn01`, `cn02`, `cn03` и `cn04`.

На все виртуальные машины был установлен дистрибутив `openSUSE Leap`. При установке была выбрана минимальная консольная конфигурация без графической среды — графический интерфейс не нужен для работы служб `SLURM`, `MUNGE` и `SSH`, но занимает дисковое пространство и оперативную память.

Каждой виртуальной машине был выделен один виртуальный процессор и 2 ГБ оперативной памяти. Первоначально для системного диска рассматривался объём 8–10 ГБ, однако установщик `openSUSE Leap` требовал не менее 12,5 ГиБ под корневой раздел и дополнительно 1–2 ГиБ под раздел подкачки. После этого виртуальные машины были пересозданы с диском объёмом 16 ГБ.

В результате была подготовлена однородная виртуальная среда: все узлы используют один дистрибутив, одинаковые аппаратные ограничения и минимальный набор служб. Сначала установка и базовая настройка были выполнены на управляющем узле, после чего тот же порядок действий был повторён на вычислительных узлах.

2.2 Настройка сети и доступа

Для каждой виртуальной машины использовались два сетевых адаптера. Первый адаптер был оставлен в режиме NAT. Он нужен для доступа к внешним репозиториям `openSUSE` и установки пакетов через `zypper`. Второй адаптер был подключён к внутренней сети VirtualBox. Именно через этот адаптер организована локальная связь узлов кластера между собой.

Для внутренней сети был выбран диапазон `10.1.0.0/24`. Управляющему узлу назначен адрес `10.1.0.21`, вычислительным узлам назначены адреса `10.1.0.31`, `10.1.0.32`, `10.1.0.33` и `10.1.0.34`. Адреса управляюще-

го и вычислительных узлов разнесены по разным десяткам, поэтому конфигурация остаётся читаемой и допускает добавление новых узлов без изменения выбранной схемы.

Разрешение имён выполнено через файл `/etc/hosts`. На всех узлах были добавлены одинаковые записи для имён `mgmt`, `cn01`, `cn02`, `cn03` и `cn04`. Недостатком такого подхода является необходимость синхронизировать файл на всех узлах, однако при пяти машинах это ограничение несущественно.

Следующим этапом была настройка SSH-доступа без ввода пароля. Для этого на каждом узле была создана SSH-пара ключей, публичные ключи всех узлов были добавлены в файл `/etc/authorized/keys` на каждом узле. Дополнительно был создан пользовательский SSH-конфиг, в котором имена узлов сопоставлены с соответствующими хостами и выбранным приватным ключом. После настройки команда вида `ssh cn01` выполняется с любого узла без указания ключа и без ввода пароля.

Итоговая сетевая схема разделяет внешнее и внутреннее взаимодействие. Доступ с хостовой машины выполняется через проброшенные NAT-порты, а обмен между узлами кластера идёт по внутренней сети VirtualBox. Поэтому для работы SLURM и SSH внутри кластера не требуется открывать отдельные внешние порты.

2.3 Установка MUNGE и SLURM

Для работы SLURM требуется единый механизм аутентификации сообщений между узлами. Эту функцию выполняет MUNGE. На управляющем узле был создан общий ключ MUNGE, после чего он был скопирован на все вычислительные узлы. Для ключа были выставлены ограниченные права доступа и владелец `munge`; одинаковый ключ на всех узлах является условием корректного обмена служебными сообщениями SLURM.

Пакеты устанавливались через пакетный менеджер `zypper`. На управляющем узле используется служба `slurmctld`, которая хранит состояние очереди, принимает задания и назначает ресурсы. На вычислительных узлах используется служба `slurmd`, которая сообщает управляющему узлу о состоянии узла и выполняет назначенные задания.

Конфигурация SLURM была сформирована через официальный конфи-

гуратор SchedMD. В конфигурации указан кластер `practice`, управляющий узел `mgmt`, вычислительные узлы `cn[01-04]` и очередь `debug`. Для выбора процесса отслеживания был оставлен вариант `proctrack/cgroup`; для выбора ресурсов использован `select/cons_tres`; для планировщика выбран `sched/backfill`. Эти параметры соответствуют рекомендациям конфигура- тора и позволяют SLURM учитывать выделенные ресурсы, отслеживать про- цессы заданий и заполнять свободные промежутки в расписании.

Один и тот же файл `slurm.conf` был размещён на всех узлах в каталоге `/etc/slurm`. На управляющем узле был создан каталог сохранения состоя- ния контроллера, на вычислительных узлах был создан каталог для состоя- ния `slurmd`. После запуска служб состояние кластера проверялось команда- ми `sinfo` и `squeue`; корректной исходной точкой для эксперимента является состояние вычислительных узлов `idle`.

2.4 Обработка исходного набора данных

Индивидуальный вариант определяет три параметра: дистрибутив `openSUSE Leap`, UID `50728` и имя задания `qe7`. Исходный набор данных был распакован и отфильтрован по UID и имени задания. В результате сформи- рован файл `acct_0921-0923_uid50728_qe7`, содержащий 1563 записи.

В выбранном фрагменте 1542 записи имеют состояние `COMPLETED`, 11 записей имеют состояние `FAILED`, 10 записей имеют состояние `CANCELLED`. Для построения распределений использовались завершённые задания, так как только для них фактическое время выполнения можно интерпретировать как корректную длительность работы задачи.

Фактическое время выполнения задания берётся из поля `ElapsedRaw`. Ошибка прогноза времени вычисляется как разность между запрошенным временем и фактическим временем:

$$\Delta T = \text{TimelimitRaw} \cdot 60 - \text{ElapsedRaw}.$$

В выбранном наборе данных минимальное фактическое время завер- шённого задания составляет 699 секунд, медиана составляет 1652 секунды, среднее значение составляет 1712,39 секунды. Максимальное значение рав- но 16660 секунд, но оно относится к хвосту распределения и не использу- ется как типичное значение при аппроксимации. Ошибка прогноза времени

для завершённых заданий имеет медиану 19948 секунд и среднее значение 19887,61 секунды; следовательно, пользовательские задания в данном варианте обычно запрашивали время с большим запасом относительно фактической длительности.

На рисунке 1 приведена сводная визуализация исходного набора данных. Она фиксирует объём индивидуальной выборки, распределение состояний и характеристики времени выполнения. Эта проверка нужна перед генерацией, потому что параметры модели должны оцениваться по заданиям варианта, а не по всему исходному набору данных.

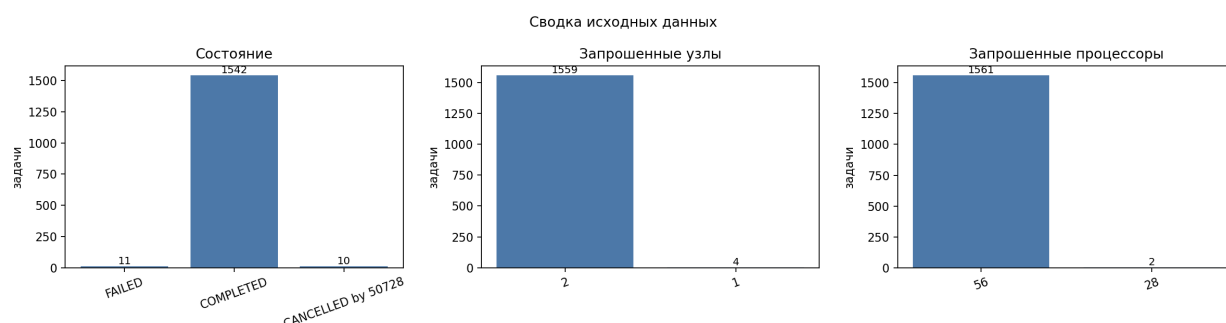


Рисунок 1 – Сводка по исходному набору данных

2.5 Аппроксимация исходных распределений

Анализ фактических длительностей показал, что в данных присутствуют две выраженные группы заданий. Первая группа сосредоточена около 1647 секунд и имеет вес около 0,809, вторая группа сосредоточена около 1925 секунд и имеет вес около 0,191. Поэтому фактическая длительность была описана смесью двух нормальных распределений. Параметры смеси приведены в таблице 1.

Таблица 1 – Параметры смеси нормальных распределений фактического времени

Компонента	Среднее, с	Стандартное отклонение, с	Вес
1	1647,26	19,89	0,809
2	1924,82	18,70	0,191

На рисунке 2 показана гистограмма исходных фактических длительностей и наложенная аппроксимация смесью нормальных распределений. Две группы заданий разделены заметным промежутком, поэтому одна нормаль-

ная компонента сгладила бы структуру данных и дала бы неверное положение пика. Смесь из двух компонент сохраняет обе характерные длительности.

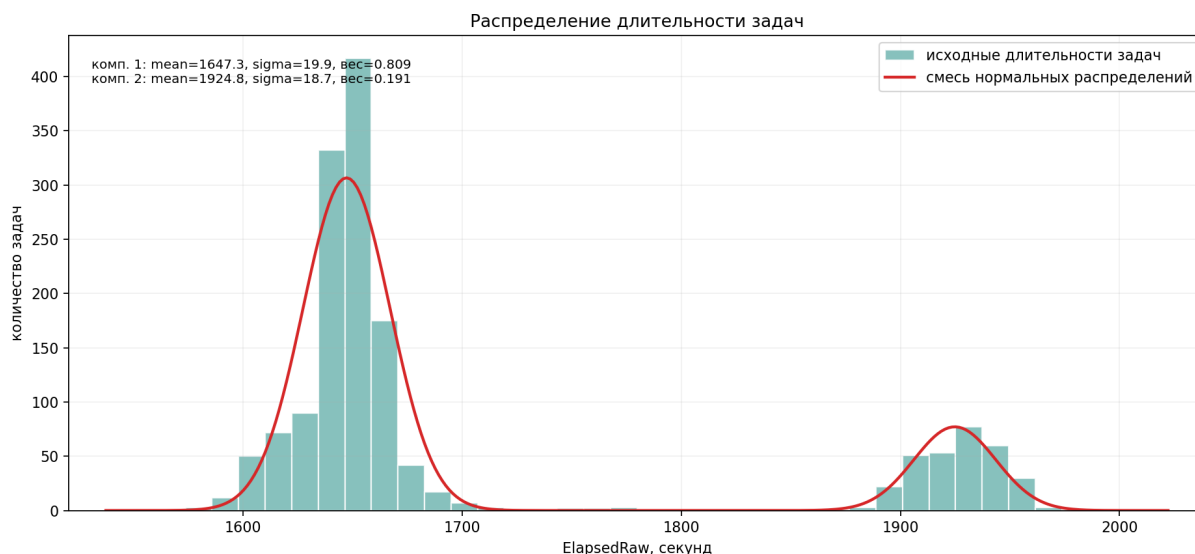


Рисунок 2 – Аппроксимация распределения фактического времени выполнения

Ошибка прогноза времени также имеет две близкие, но различимые группы. Так как ошибка является положительной величиной и рассматривается как мультипликативное отклонение, для неё используется логнормальная модель. Параметры смеси логнормальных распределений приведены в таблице 2.

Таблица 2 – Параметры смеси логнормальных распределений ошибки прогноза

Компонента	μ	σ	Вес
1	9,8871	0,0014	0,192
2	9,9011	0,0014	0,808

На рисунке 3 показано распределение ошибки прогноза времени в масштабированной шкале эксперимента. Голубая гистограмма соответствует расчётной ошибке из манифеста, оранжевая гистограмма показывает фактическую ошибку после выполнения заданий в SLURM, а красная кривая задаёт смесь логнормальных распределений, использованную генератором. Наложение фактических значений на расчётные позволяет проверить, насколько

выполнение в виртуальном кластере смещает исходную модель. В текущем запуске медиана расчётной и фактической ошибки совпала и составила около 897 секунд, а фактические значения оказались немного сдвинуты из-за накладных расходов запуска.

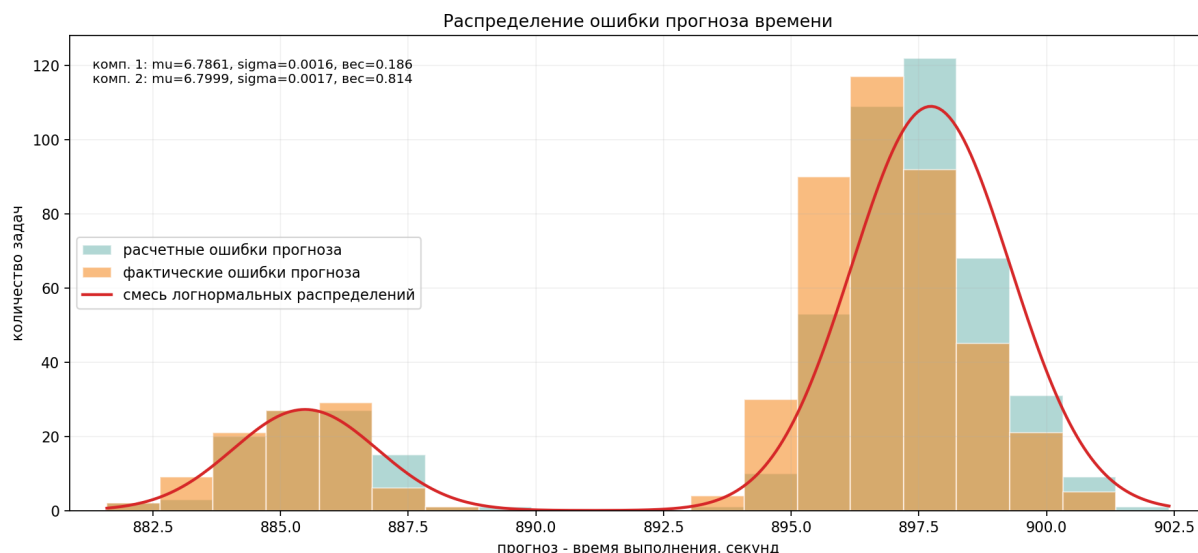


Рисунок 3 – Распределение расчётной и фактической ошибки прогноза времени

На рисунке 4 та же ошибка показана после логарифмирования. Такой график нужен для проверки выбранной логнормальной модели: если величина имеет логнормальное распределение, то её логарифм должен быть близок к нормальному распределению или смеси нормальных распределений. В логарифмической шкале две компоненты разделяются отчётливее, чем в исходной шкале секунд.

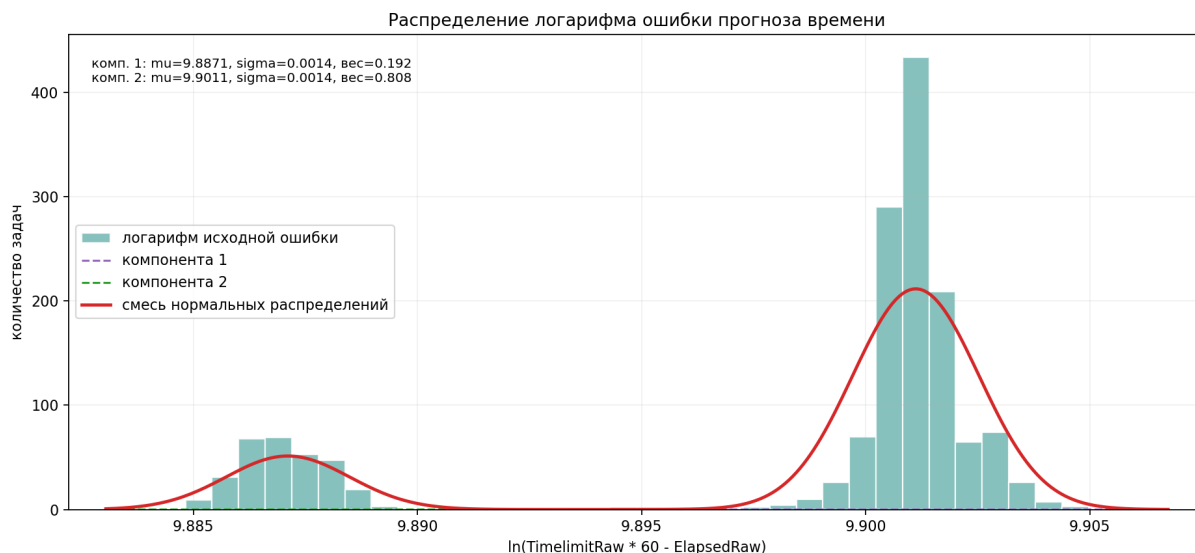


Рисунок 4 – Распределение логарифма ошибки прогноза времени

2.6 Генерация пакетных заданий

Генератор запускается на управляющем узле `mgmt`. На вход ему передаются число заданий, масштаб фактического времени, масштаб планового времени и параметры распределений. Для каждого задания генератор выбирает фактическую длительность по смеси нормальных распределений, выбирает ошибку прогноза по смеси логнормальных распределений, рассчитывает плановое время как сумму фактической длительности и ошибки прогноза, после чего масштабирует оба значения для практического запуска на учебном кластере.

Масштабирование необходимо, потому что исходные задания длятся десятки минут, а полный эксперимент на сотнях заданий без масштабирования занял бы слишком много времени. При этом масштабируются и фактическое время, и плановое время, поэтому относительная ошибка сохраняется. В последнем сформированном манифесте использовался масштаб около 0,045. После масштабирования медиана фактического времени `sleep` составила 74 секунды, среднее значение составило 76,116 секунды, максимум составил 89 секунд. Медиана планового времени составила 972 секунды.

Количество узлов для задания выбирается по нормальному распределению с ограничением от 1 до 4. Такой способ даёт больше заданий на 2 узла, меньше заданий на 1 и 3 узла и редкие задания на 4 узла. В манифесте на 500 заданий распределение числа узлов получилось таким, как показано в табли-

це 3.

Таблица 3 – Распределение сгенерированных заданий по числу узлов

Число узлов	Число заданий
1	118
2	245
3	124
4	13

На рисунке 5 сравниваются исходное распределение фактических длительностей и сгенерированное распределение. Сгенерированная выборка сохраняет две основные группы заданий, но не совпадает с исходной гистограммой дословно. Это ожидаемый результат: генератор создаёт новую случайную выборку конечного размера, а не копирует строки исходного файла.

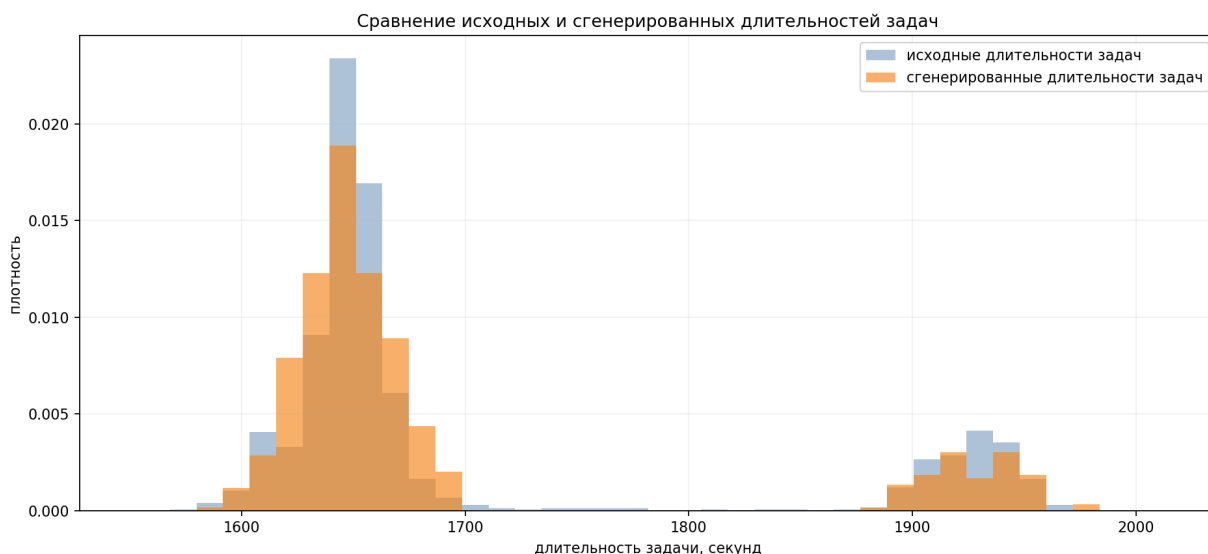


Рисунок 5 – Сравнение исходного и сгенерированного времени выполнения

2.7 Запуск заданий и сбор фактической статистики

После генерации пакетные файлы отправляются в SLURM командой `sbatch`. Состояние очереди отслеживается через `squeue`, а сведения о завершённых заданиях собираются в файл `results.csv`. Для каждого задания сохраняются идентификатор SLURM, имя сценария, состояние, узел выполнения, время ожидания в очереди, фактическое время выполнения и наблюдаемое полное время.

При анализе важно разделять три величины. Первая величина — запланированное время из манифеста генератора. Вторая — значение `--time`, передаваемое в SLURM. SLURM округляет его до минут, поэтому оно не подходит для точного расчёта ошибки на коротких масштабированных заданиях. Третья величина — фактическое время выполнения, полученное из статистики SLURM. Поэтому для расчёта фактической ошибки используется плановое значение из `manifest.csv`, а не округлённое значение из очереди.

Полностью завершённый эксперимент на 500 заданиях показал, что все задания завершились в состоянии `COMPLETED`. Запланированное время `sleep` находилось в диапазоне от 72 до 89 секунд, медиана составила 74 секунды. Фактическое время выполнения находилось в диапазоне от 72 до 90 секунд, медиана составила 75 секунд. Накладные расходы относительно команды `sleep` имели медиану около 0,562%, среднее значение около 0,656%, а максимальное значение около 1,389%. При таком масштабе постоянная задержка запуска и завершения задания остаётся малой относительно длительности самих заданий.

На рисунке 6 показано сравнение планового и фактического времени по отдельным заданиям. Плановое время заметно больше фактического, что соответствует исходной задаче моделирования: пользователь заранее запрашивает лимит времени с большим запасом, а реальное выполнение завершается раньше. Фактическое время меняется в узком диапазоне, так как масштабированные задания выполняют только команду `sleep`.



Рисунок 6 — Плановое и фактическое время выполнения заданий

На рисунке 7 показано отклонение фактического времени от времени

sleep. Этот график позволяет оценить накладные расходы SLURM и виртуальной среды по отдельным заданиям. Для финального запуска был выбран такой масштаб времени, при котором даже постоянная задержка в одну секунду не приводит к большому относительному отклонению: медианные значения по группам остаются существенно ниже контрольного порога 3%.



Рисунок 7 – Накладные расходы фактического выполнения по заданиям

2.8 Анализ фактической ошибки прогноза

После получения фактического времени выполнения можно пересчитать ошибку прогноза уже не для исходных данных, а для выполненных на кластере заданий. Эта величина определяется как разность между масштабированным плановым временем из манифеста и фактическим временем выполнения, полученным из SLURM.

На рисунке 8 показано распределение фактической ошибки прогноза. В завершённом запуске на 500 заданий ошибка находилась в диапазоне от 881 до 901 секунды, медиана составила около 897 секунд. Форма распределения остаётся компактной, потому что плановое время было сгенерировано из той же модели, а фактическое время отличается от sleep в основном на небольшую постоянную задержку. Наложённая кривая на этом графике является аппроксимацией наблюдаемой фактической ошибки, а не новой моделью генерации.

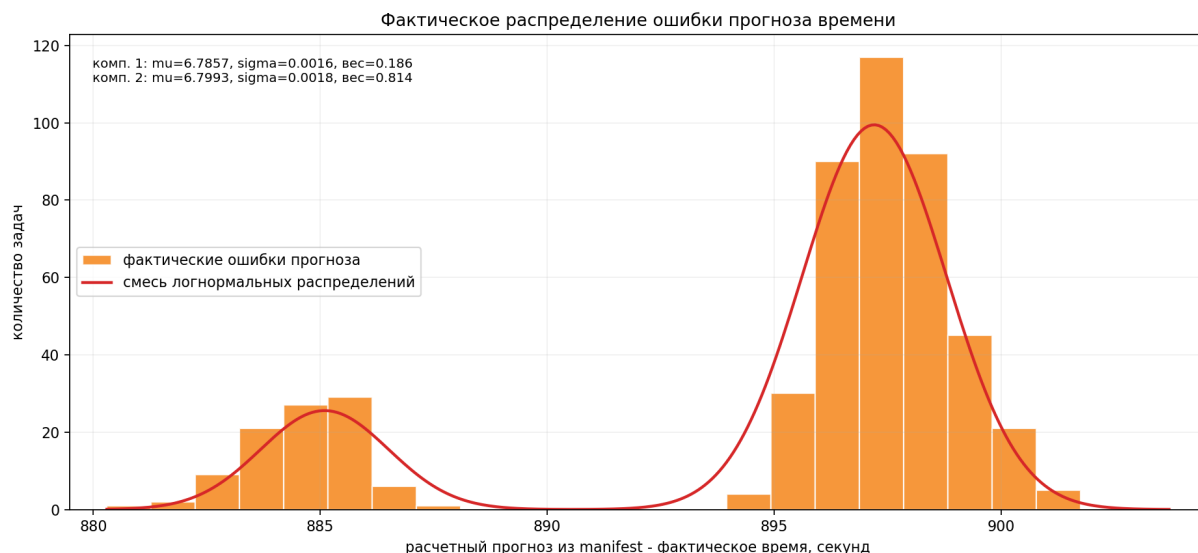


Рисунок 8 – Распределение фактической ошибки прогноза времени

На рисунке 9 показан логарифм фактической ошибки прогноза. Как и для исходных данных, логарифмическая шкала используется для проверки логнормальной природы ошибки. Наложенная кривая показывает, что после выполнения на кластере ошибка сохраняет форму, близкую к той, которая была заложена генератором, но становится немного сдвинутой из-за накладных расходов выполнения.

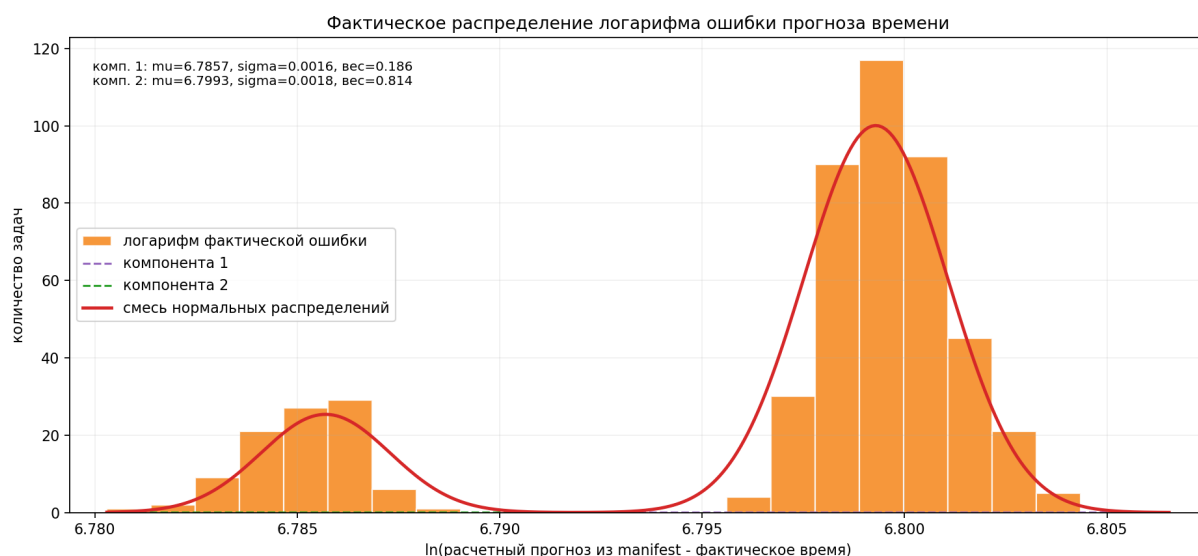


Рисунок 9 – Распределение логарифма фактической ошибки прогноза времени

Итоговый запуск на 500 заданий подтвердил выбранный масштаб вре-

мени. При четырёх вычислительных узлах эксперимент остаётся выполнимым за ограниченное время, а относительное влияние накладных расходов не искажает распределения: медианное отклонение фактического времени от `sleep` составило около 0,562%, среднее значение около 0,656%, максимальное значение около 1,389%. Фактическая ошибка прогноза поэтому остаётся близкой к расчётной ошибке из манифеста; это сопоставление показано на рисунке 3.

Заключение

В ходе выполнения летней практики была подготовлена учебная среда для моделирования работы очереди вычислительного кластера. На локальной машине были созданы пять виртуальных машин с openSUSE Leap: управляющий узел `mgmt` и вычислительные узлы `cn01–cn04`. Для узлов настроены статические IP-адреса, разрешение имён, SSH-доступ по ключам и общая конфигурация MUNGE.

На кластере установлен SLURM. Конфигурационный файл сформирован через официальный конфигуратор и синхронизирован между всеми узлами. Управляющий узел запускает службу `slurmctld`, вычислительные узлы запускают службу `slurmd`. Работоспособность кластера проверялась командами `sinfo`, `squeue` и запуском пакетных заданий.

Для индивидуального варианта был подготовлен фрагмент исходного набора данных с UID 50728 и именем задания `qe7`. По выбранным данным рассчитаны параметры распределения фактического времени выполнения и ошибки прогноза времени. Генератор формирует пакетные сценарии SLURM, выбирает количество узлов, рассчитывает плановое и фактическое время задания и отправляет задания в очередь. Аналитический модуль строит графики исходного распределения, сгенерированного распределения и фактических результатов выполнения.

Последний сформированный манифест содержит 500 заданий. Распределение числа узлов по заданиям составило 118 заданий на один узел, 245 заданий на два узла, 124 задания на три узла и 13 заданий на четыре узла. При выбранном масштабе времени медианное плановое время составляет 972 секунды, медианное время `sleep` составляет 74 секунды. Оценка длительности полного запуска на четырёх вычислительных узлах составляет около 6,5 часов, а ожидаемое отклонение фактического времени от планируемого при накладных расходах порядка 1,5 секунды на задание составляет около 2%.

Таким образом, поставленная задача выполнена: учебный кластер SLURM развёрнут, генератор пользовательских заявок реализован, исходные данные обработаны, а результаты генерации и выполнения представлены в виде численных характеристик и графиков.

Список использованных источников

- [1] Slurm Workload Manager Documentation [Электронный ресурс] // SchedMD. URL: <https://slurm.schedmd.com/documentation.html> (дата обращения: 26.06.2026).
- [2] Slurm Configuration Tool [Электронный ресурс] // SchedMD. URL: <https://slurm.schedmd.com/configurator.html> (дата обращения: 26.06.2026).
- [3] MUNGE Uid 'N' Gid Emporium [Электронный ресурс]. URL: <https://dun.github.io/munge/> (дата обращения: 26.06.2026).
- [4] openSUSE Leap Documentation [Электронный ресурс] // openSUSE. URL: <https://doc.opensuse.org/> (дата обращения: 26.06.2026).
- [5] Oracle VM VirtualBox User Manual [Электронный ресурс] // Oracle. URL: <https://www.virtualbox.org/manual/> (дата обращения: 26.06.2026).
- [6] ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. М.: Стандартинформ, 2017.

Приложение А. Фрагмент конфигурации SLURM

```
1  ClusterName=practice
2  SlurmctldHost=mgmt
3  ProctrackType=proctrack/cgroup
4  ReturnToService=2
5  SlurmctldPidFile=/var/run/slurmctld.pid
6  SlurmctldPort=6817
7  SlurmdPidFile=/var/run/slurmd.pid
8  SlurmdPort=6818
9  SlurmdSpoolDir=/var/spool/slurmd
10 SlurmUser=slurm
11 StateSaveLocation=/var/spool/slurmctld
12 TaskPlugin=task/affinity,task/cgroup
13 SchedulerType=sched/backfill
14 SelectType=select/cons_tres
15 JobCompType=jobcomp/none
16 SlurmctldLogFile=/var/log/slurmctld.log
17 SlurmdLogFile=/var/log/slurmd.log
18 NodeName=cn[01-04] NodeAddr=cn[01-04] CPUs=1 State=UNKNOWN
19 PartitionName=debug Nodes=ALL Default=YES MaxTime=INFINITE State=UP
```

Приложение Б. Пример сгенерированного задания SLURM

```
1  #!/usr/bin/env bash
2  #SBATCH --job-name=ge7_0001
3  #SBATCH --partition=debug
4  #SBATCH --nodes=2
5  #SBATCH --ntasks=2
6  #SBATCH --time=00:17:00
7  #SBATCH --output=generated/logs/%x-%j.out
8  #SBATCH --error=generated/logs/%x-%j.err
9
10 set -euo pipefail
11
12 mkdir -p "$(dirname "generated/logs/generated-jobs.log")"
13 start_time="$(date --iso-8601=seconds)"
14 echo "job_index=1 source_job_id=507280001 start=${start_time} planned_sleep=74" >>
15   ↪ "generated/logs/generated-jobs.log"
16 sleep 74
17 end_time="$(date --iso-8601=seconds)"
18 echo "job_index=1 source_job_id=507280001 end=${end_time} planned_sleep=74" >>
19   ↪ "generated/logs/generated-jobs.log"
```

Приложение В. Код генератора пакетных заданий

Файл `generate/main.py` содержит разбор параметров, выбор значений из распределений, масштабирование времени и запись манифеста.

```
1 from __future__ import annotations
2
3 import argparse
4 import csv
5 import math
6 import random
7 import shutil
8 from pathlib import Path
9 from generate.template import render_batch_script
10
11
12
13 def positive_int(value: str) -> int:
14     parsed = int(value)
15     if parsed <= 0:
16         raise argparse.ArgumentTypeError("value must be positive")
17     return parsed
18
19
20 def positive_float(value: str) -> float:
21     parsed = float(value)
22     if parsed <= 0:
23         raise argparse.ArgumentTypeError("value must be positive")
24     return parsed
25
26
27 def parse_component(value: str) -> tuple[float, float, float]:
28     parts = value.split(",")
29     if len(parts) != 3:
30         raise argparse.ArgumentTypeError("component must be center,sigma,weight")
31     center = float(parts[0])
32     sigma = float(parts[1])
33     weight = float(parts[2])
34     if sigma <= 0 or weight <= 0:
35         raise argparse.ArgumentTypeError("sigma and weight must be positive")
36     return center, sigma, weight
37
38
39 def choose_component(rng: random.Random, components: list[tuple[float, float, float]]) ->
40     ↪ tuple[float, float]:
41     total = sum(weight for _, _, weight in components)
42     roll = rng.random() * total
43     cumulative = 0.0
44     for center, sigma, weight in components:
45         cumulative += weight
46         if roll <= cumulative:
47             return center, sigma
48     center, sigma, _ = components[-1]
49     return center, sigma
50
51 def sample_lognormal_value(rng: random.Random, components: list[tuple[float, float, float]])
52     ↪ -> int:
53     mu, sigma = choose_component(rng, components)
54     return max(1, int(round(rng.lognormvariate(mu, sigma))))
55
56 def sample_normal_value(rng: random.Random, components: list[tuple[float, float, float]]) ->
57     ↪ int:
58     mean, sigma = choose_component(rng, components)
59     return max(1, int(round(rng.normalvariate(mean, sigma))))
60
61 def sample_node_count(rng: random.Random) -> int:
62     sampled = int(round(rng.normalvariate(2.0, 0.75)))
63     return min(4, max(1, sampled))
64
65
66 def format_slurm_time(total_seconds: int) -> str:
```

```

67     total_seconds = max(60, int(math.ceil(total_seconds / 60.0) * 60))
68     days, remainder = divmod(total_seconds, 24 * 60 * 60)
69     hours, remainder = divmod(remainder, 60 * 60)
70     minutes, seconds = divmod(remainder, 60)
71     if days:
72         return f"{days}-{hours:02d}:{minutes:02d}:{seconds:02d}"
73     return f"{hours:02d}:{minutes:02d}:{seconds:02d}"
74
75
76 def clean_run_dir(run_dir: Path) -> None:
77     if run_dir.exists():
78         shutil.rmtree(run_dir)
79     (run_dir / "slurm_jobs").mkdir(parents=True)
80
81
82 def generate_jobs(
83     run_dir: Path,
84     count: int,
85     sleep_scale: float,
86     time_scale: float,
87     seed: int,
88     partition: str,
89     runtime_components: list[tuple[float, float, float]],
90     error_components: list[tuple[float, float, float]],
91 ) -> Path:
92     clean_run_dir(run_dir)
93     rng = random.Random(seed)
94     slurm_jobs_dir = run_dir / "slurm_jobs"
95     manifest_path = run_dir / "manifest.csv"
96     log_file = Path("generated/logs/generated-jobs.log")
97
98     with manifest_path.open("w", newline="", encoding="utf-8") as manifest_file:
99         writer = csv.DictWriter(
100             manifest_file,
101             fieldnames=[
102                 "script",
103                 "source_job_id",
104                 "source_state",
105                 "source_elapsed_seconds",
106                 "sampled_elapsed_seconds",
107                 "sampled_error_seconds",
108                 "generated_forecast_seconds",
109                 "scaled_sleep_seconds",
110                 "scaled_forecast_seconds",
111                 "slurm_time",
112                 "nodes",
113                 "ntasks",
114             ],
115         )
116         writer.writeheader()
117
118         for index in range(1, count + 1):
119             sampled_elapsed_seconds = sample_normal_value(
120                 rng, runtime_components
121             )
122             sampled_error_seconds = sample_lognormal_value(rng, error_components)
123             generated_forecast_seconds = sampled_elapsed_seconds + sampled_error_seconds
124             scaled_sleep_seconds = max(1, int(round(sampled_elapsed_seconds * sleep_scale)))
125             scaled_forecast_seconds = max(
126                 60,
127                 int(round(generated_forecast_seconds * time_scale)),
128             )
129             nodes = sample_node_count(rng)
130             ntasks = nodes
131             slurm_time = format_slurm_time(scaled_forecast_seconds)
132             source_job_id = str(507280000 + index)
133             script_name = f"job_{index:04d}_{source_job_id}.slurm"
134             script_path = slurm_jobs_dir / script_name
135
136             script_path.write_text(
137                 render_batch_script(
138                     index=index,
139                     source_job_id=source_job_id,
140                     partition=partition,
141                     nodes=nodes,
142                     ntasks=ntasks,
143                     slurm_time=slurm_time,
144                     sleep_seconds=scaled_sleep_seconds,

```

```

145         log_file=log_file,
146     ),
147     encoding="utf-8",
148 )
149 script_path.chmod(0o755)
150
151 writer.writerow(
152     {
153         "script": script_name,
154         "source_job_id": source_job_id,
155         "source_state": "GENERATED",
156         "source_elapsed_seconds": sampled_elapsed_seconds,
157         "sampled_elapsed_seconds": sampled_elapsed_seconds,
158         "sampled_error_seconds": sampled_error_seconds,
159         "generated_forecast_seconds": generated_forecast_seconds,
160         "scaled_sleep_seconds": scaled_sleep_seconds,
161         "scaled_forecast_seconds": scaled_forecast_seconds,
162         "slurm_time": slurm_time,
163         "nodes": nodes,
164         "ntasks": ntasks,
165     }
166 )
167
168 return manifest_path
169
170
171 def parse_args() -> argparse.Namespace:
172     parser = argparse.ArgumentParser(description="Generate Slurm jobs on the mgmt VM.")
173     parser.add_argument("--output-root", type=Path, default=Path("generated/cluster_runs"))
174     parser.add_argument("--run-id", required=True)
175     parser.add_argument("--count", type=positive_int, default=200)
176     parser.add_argument("--sleep-scale", type=positive_float, default=0.01)
177     parser.add_argument("--time-scale", type=positive_float, default=0.01)
178     parser.add_argument("--seed", type=int, default=50728)
179     parser.add_argument("--partition", default="debug")
180     parser.add_argument(
181         "--runtime-normal-component",
182         action="append",
183         type=parse_component,
184         default=[],
185         help="Runtime normal component as mean,sigma,weight.",
186     )
187     parser.add_argument(
188         "--error-component",
189         action="append",
190         type=parse_component,
191         default=[],
192         help="Forecast error lognormal component as mu,sigma,weight.",
193     )
194     return parser.parse_args()
195
196
197 def main() -> None:
198     args = parse_args()
199     run_dir = (args.output_root / args.run_id).resolve()
200     runtime_components = args.runtime_normal_component or [(1701.49, 435.21, 1.0)]
201     error_components = args.error_component or [(9.8981, 0.0371, 1.0)]
202     generate_jobs(
203         run_dir=run_dir,
204         count=args.count,
205         sleep_scale=args.sleep_scale,
206         time_scale=args.time_scale,
207         seed=args.seed,
208         partition=args.partition,
209         runtime_components=runtime_components,
210         error_components=error_components,
211     )
212     print(run_dir)
213
214
215 if __name__ == "__main__":
216     main()

```

Файл `generate/template.py` формирует текст пакетного файла SLURM для отдельного задания.


```

1  from __future__ import annotations
2
3  from pathlib import Path
4
5
6  def render_batch_script(
7      index: int,
8      source_job_id: str,
9      partition: str,
10     nodes: int,
11     ntasks: int,
12     slurm_time: str,
13     sleep_seconds: int,
14     log_file: Path,
15 ) -> str:
16     return f"""#!/usr/bin/env bash
17 #SBATCH --job-name=qe7_{index:04d}
18 #SBATCH --partition={partition}
19 #SBATCH --nodes={nodes}
20 #SBATCH --ntasks={ntasks}
21 #SBATCH --time={slurm_time}
22 #SBATCH --output=generated/logs/%x-%j.out
23 #SBATCH --error=generated/logs/%x-%j.err
24
25 set -euo pipefail
26
27 mkdir -p "$(dirname "{log_file}")"
28 start_time="$(date --iso-8601=seconds)"
29 echo "job_index={index} source_job_id={source_job_id} start=${{start_time}}
   ↳ planned_sleep={sleep_seconds}" >> "{log_file}"
30 sleep {sleep_seconds}
31 end_time="$(date --iso-8601=seconds)"
32 echo "job_index={index} source_job_id={source_job_id} end=${{end_time}}
   ↳ planned_sleep={sleep_seconds}" >> "{log_file}"
33 """

```